

INFORME FINAL – TRABAJO INTEGRADOR

1. Integrantes y roles

El proyecto fue desarrollado por los siguientes integrantes:

Integrante	Rol
Fernando Chacón	Diseño del sistema – UML – Base de Datos – Estructura de paquetes
Ciro Giorgini	DAO – Services – Validaciones – Reglas de Negocio – Manejo de transacciones
Francisco Frascona	AppMenu – Interfaz de consola – Pruebas – Informe y video

2. Dominio elegido y justificación

El dominio seleccionado fue **Empleado → Legajo**, donde cada empleado posee un único legajo administrativo. Esta elección se basa en los siguientes fundamentos:

Modelo natural para una relación 1→1

En organizaciones reales, cada empleado tiene exactamente **un** legajo. Esto permite aplicar de manera clara una relación **uno a uno** sin ambigüedad.

Simplicidad y consistencia

El dominio permite mantener un diseño limpio y entendible, ideal para aplicar:

- Relación 1→1 unidireccional
- Reglas de negocio específicas
- Validaciones obligatorias
- Integridad referencial estricta

Buen candidato para el uso de transacciones

La creación o eliminación simultánea de empleado + legajo requiere que ambas operaciones se completen juntas, lo cual permite demostrar el uso de:

- `commit()`
- `rollback()`
- Manejo de errores en cascada

Complejidad adecuada para el TFI

El dominio tiene la cantidad justa de entidades, atributos y reglas para demostrar dominio de:

- Modelado UML
- BD relacional
- DAO/Service
- Consola interactiva
- Transacciones y validaciones

3. Diseño: decisiones clave + UML

Relación 1→1 unidireccional

El diseño establece que:

- **Empleado** conoce a **Legajo** mediante un atributo privado.
- **Legajo** no referencia a Empleado en el modelo Java (unidireccionalidad).
- La base de datos se encarga de garantizar la unión 1→1.

Implementación en BD

El proyecto utiliza una **FK única** en la tabla de legajo, de modo que cada empleado puede aparecer como máximo una vez.

Esto permite:

- Evitar duplicación de legajos
- Mantener integridad referencial
- Garantizar la relación uno a uno desde la base

Atributos principales en el diseño UML

Empleado incluye atributos como:

- identificador
- marca de eliminación lógica
- nombre, apellido
- DNI
- email
- área
- fecha de ingreso
- relación hacia un legajo

Legajo incluye atributos como:

- identificador
- marca de eliminación lógica
- número de legajo
- categoría
- estado (enum)
- fecha de alta
- observaciones

Además, el enum **Estado** define los valores *ACTIVO* e *INACTIVO*.

Métodos incluidos

Ambas entidades incluyen:

- Constructores
- Métodos de acceso (getters/setters)
- Método de representación textual
- Métodos auxiliares para manipulación de atributos

(Aquí el grupo debe insertar su UML como imagen.)

4. Arquitectura por capas (paquetes)

El proyecto se estructura en distintas capas, cada una con responsabilidades claras:

config/

Contiene la clase encargada de administrar la conexión a la base de datos.

Funciones principales:

- Leer propiedades de conexión
- Crear y devolver una conexión reutilizable
- Manejar excepciones de conexión

entities/

Incluye las clases del dominio:

- Empleado
- Legajo
- Estado (enum)

Cada clase encapsula atributos, reglas básicas y comportamiento propio del modelo.

dao/

Incluye:

- Interfaces DAO genéricas
- Implementaciones concretas para cada entidad
- Métodos para operaciones CRUD (crear, buscar, actualizar, eliminar de forma lógica)
- Uso de consultas preparadas
- Conversión entre registros de BD y objetos del modelo

Funciones típicas implementadas:

- buscarPorId
- buscarTodos
- guardar
- actualizar
- baja lógica
- búsquedas por atributos clave (ej. DNI, número de legajo)

service/

Capa donde se aplican **validaciones y reglas de negocio**.

Incluye:

- Servicios de Empleado
- Servicios de Legajo
- Métodos que coordinan varias operaciones DAO
- Implementación de transacciones

Funciones típicas:

- crearEmpleadoConLegajo
- eliminarEmpleadoYLegajo
- validar existencia de DNI
- validar existencia de número de legajo
- impedir legajos duplicados
- manejar commit/rollback

app/

Contiene:

- El menú interactivo del sistema
- Funciones de ingreso de datos por consola
- Validaciones de entrada
- Llamadas a los Services
- Mensajes informativos al usuario

5. Persistencia y transacciones

Estructura de la base de datos

La base contiene 2 tablas principales:

- **empleado**
- **legajo**

Ambas incluyen:

- Identificador numérico
- Atributo de eliminación lógica
- Campos propios de cada entidad
- Fechas de creación y modificación

La tabla de legajo contiene una clave foránea hacia empleado, con restricción de unicidad para asegurar el 1→1.

Restricción del 1→1

La integridad se garantiza mediante:

- FK hacia empleado
- Restricción UNIQUE sobre esa FK
- Eliminación lógica para evitar perder registros históricos

Operaciones transaccionales

Las operaciones que requieren transacción son:

- Crear empleado + crear legajo
- Asociar legajo a empleado
- Eliminar empleado + eliminar legajo
- Modificaciones encadenadas

Manejo de commit/rollback

En la capa service:

- Se desactiva el autocommit
- Se ejecutan múltiples operaciones DAO
- Si ninguna falla → commit
- Si alguna falla → rollback
- Luego se restablece el estado original de la conexión

6. Validaciones y reglas de negocio

El proyecto implementa múltiples validaciones:

Validaciones de entidad

- DNI obligatorio y único
- Número de legajo obligatorio y único
- Estado válido (enum)

- Campos obligatorios no pueden venir vacíos
- Fechas deben tener formato válido

Reglas de negocio

- Un empleado no puede tener más de un legajo
- No se permite borrar físicamente
- Siempre se debe mantener consistencia entre empleado y legajo
- No se puede crear legajo para un empleado eliminado
- No se pueden duplicar DNI ni números de legajo

Validaciones en consola

- Datos vacíos
- Opciones no válidas
- IDs inexistentes
- Formato incorrecto
- Reintentos de carga

7. Pruebas realizadas

El proyecto fue probado tanto desde SQL como desde el menú de aplicación.

Pruebas en consola (AppMenu)

Capturas incluidas en el informe:

- Alta de empleado
- Alta de legajo
- Búsqueda por DNI

- Búsqueda por número de legajo
- Listado completo
- Eliminación lógica
- Actualización de atributos
- Comprobación de estados

Pruebas SQL (capturas)

Se prueban:

- Inserciones iniciales
- Restricción UNIQUE funcionando
- Relación 1→1 validada
- Consultas JOIN entre empleado y legajo
- Verificación de eliminación lógica

Prueba de rollback

Se simula:

- Crear empleado
- Fallar la creación de legajo
- Resultado esperado: ningún registro queda en la BD

8. Conclusiones y mejoras

Conclusiones

El proyecto permitió integrar todos los contenidos de la materia:

- Diseño UML
- Modelado relacional

- Desarrollo por capas
- Uso correcto de DAO y Service
- Manejo de excepciones y transacciones
- Validaciones y reglas de negocio
- Persistencia en MySQL
- Interfaz de usuario por consola

Los integrantes pudieron comprender la importancia de la integridad referencial, el diseño por capas y la robustez en el manejo de errores.

Dificultades principales

- Implementar correctamente la relación 1→1
- Gestionar transacciones sin perder consistencia
- Evitar duplicación de legajos
- Manejar validaciones de datos de consola
- Coordinar operaciones en cascada (empleado + legajo)

Mejoras futuras

- Migrar la consola a una interfaz gráfica
- Implementar logs
- Manejar sesiones de usuario
- Añadir auditoría de modificaciones
- Agregar exportación de datos

9. Fuentes y herramientas utilizadas

Herramientas

- Java

- NetBeans
- MySQL
- MySQL Workbench

Asistencia con IA

- Se utilizó ChatGPT para organizar ideas y aclarar dudas conceptuales